

Rapport Technique : Architecture et Déploiement d'un Passthrough GPU Dynamique sous Arch Linux

L'évolution des technologies de virtualisation matérielle a permis de franchir un cap décisif dans la gestion des ressources informatiques, effaçant progressivement la nécessité des environnements en double amorçage (Dual-Boot). Le présent rapport technique détaille l'ingénierie, la configuration et le déploiement d'une architecture de virtualisation avancée basée sur le noyau Linux (KVM) et le framework VFIO (Virtual Function I/O). L'objectif est de configurer un système hôte Arch Linux capable de transférer dynamiquement une carte graphique dédiée (dGPU) vers une machine virtuelle Windows 11, tout en maintenant le système hôte actif et fonctionnel sur son processeur graphique intégré (iGPU), et ce sans nécessiter le redémarrage du serveur d'affichage Wayland.

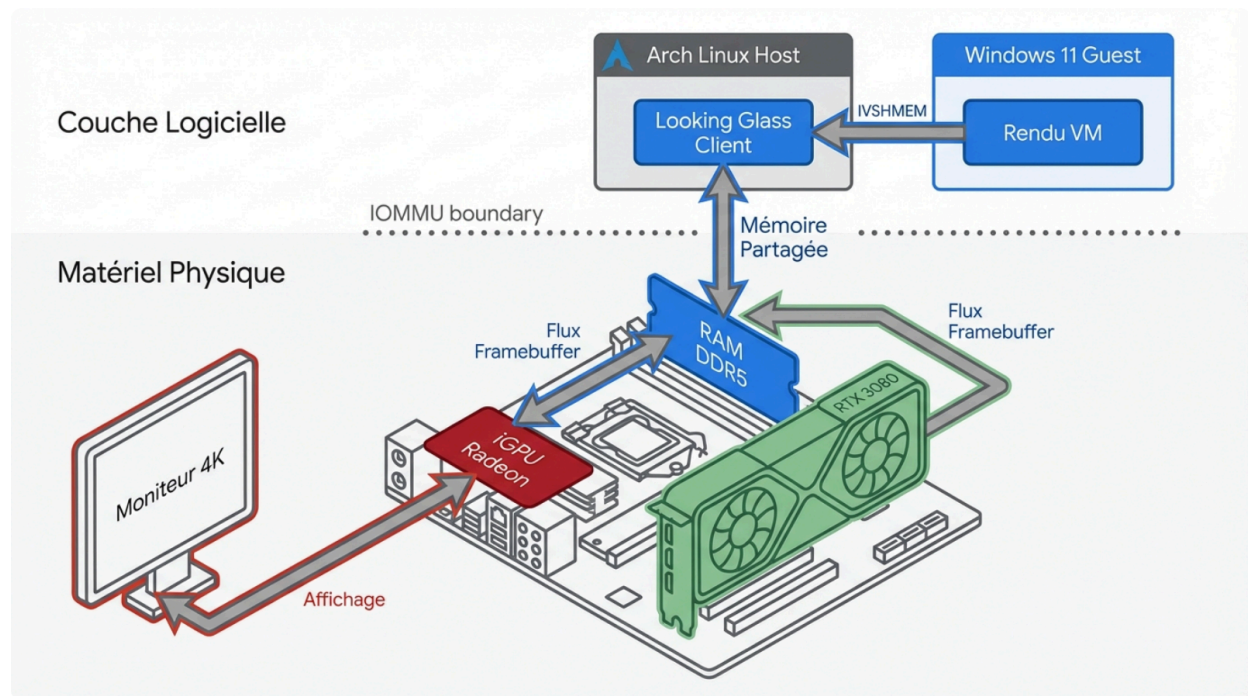
Ce document est destiné aux administrateurs systèmes et aux ingénieurs évoluant quotidiennement sous Arch Linux et Windows, possédant une familiarité conceptuelle avec les machines virtuelles, mais nécessitant une approche rigoureuse et étape par étape de l'écosystème QEMU/libvirt. L'architecture cible s'appuie sur un processeur AMD Ryzen 7 7700X (intégrant un iGPU Radeon), une carte graphique dédiée NVIDIA RTX 3080, et une carte mère MSI MAG B850 Tomahawk MAX WIFI. L'affichage physique est confié à un unique moniteur 4K (3440x1440px) connecté directement à la carte mère.

Principes Fondamentaux et Topologie Matérielle

L'architecture matérielle requise pour le passthrough PCIe (PCI Express) repose sur l'isolation stricte des composants via la technologie IOMMU (Input/Output Memory Management Unit), désignée sous le nom d'AMD-Vi chez AMD.¹ L'IOMMU permet au processeur de mapper l'espace d'adressage mémoire physique d'un périphérique PCIe directement vers l'espace mémoire virtuel d'une machine virtuelle, garantissant ainsi un accès direct (DMA) sans l'intervention de l'hyperviseur, ce qui permet d'obtenir des performances quasi-natives.

La particularité de cette implémentation réside dans son caractère dynamique. L'écran 4K étant connecté à la carte mère, l'iGPU Radeon gère en permanence le compositeur Wayland (via GDM). La RTX 3080, quant à elle, opère dans un mode hybride : elle est initialement gérée par le pilote propriétaire NVIDIA sur l'hôte Linux pour permettre le rendu déporté (PRIME Offloading) d'applications gourmandes, puis elle est isolée, déchargée de ses pilotes, et réassignée au pilote vfio-pci à la volée lors du démarrage de la machine virtuelle Windows 11.²

Topologie du Flux Vidéo et Isolation Matérielle



Le moniteur 4K est physiquement relié à la carte mère. L'iGPU gère le compositeur Wayland. La RTX 3080 effectue le rendu de la VM, dont le framebuffer est copié vers l'hôte via un périphérique mémoire partagé PCIe (IVSHMEM) pour un affichage sans latence.

La carte mère MSI MAG B850 Tomahawk MAX WIFI offre une topologie PCIe adéquate, où le premier connecteur PCIe x16 (hébergeant la RTX 3080) est généralement isolé dans son propre groupe IOMMU, une condition *sine qua non* pour transférer le périphérique sans emporter d'autres contrôleurs critiques du système hôte.

Phase 1 : Paramétrage du Firmware UEFI (BIOS)

Avant toute manipulation logicielle, le matériel sous-jacent doit être instruit pour autoriser la virtualisation et gérer correctement la mémoire massive de la carte graphique dédiée. L'interface UEFI de la carte mère MSI MAG B850 nécessite des ajustements précis.

L'accès au BIOS s'effectue traditionnellement via la touche Suppr (Del) lors de la séquence POST. Les paramètres suivants doivent être modifiés pour garantir l'activation de l'isolation matérielle et l'adressage mémoire étendu.

Catégorie du BIOS	Paramètre	Valeur Requise	Justification
-------------------	-----------	----------------	---------------

MSI			Technique
Advanced CPU Configuration	SVM Mode	Enabled	Active les extensions de virtualisation matérielle du processeur AMD Ryzen (Secure Virtual Machine), indispensables pour KVM.
Advanced CPU Configuration / AMD CBS	IOMMU	Enabled	Active l'isolation des périphériques PCI (AMD-Vi). Permet d'assigner un périphérique exclusif à la machine virtuelle sans conflit avec l'hôte. ¹
PCI Subsystem Settings	Above 4G Decoding	Enabled	Autorise le système d'exploitation à mapper l'espace d'E/S des périphériques PCIe disposant de grandes quantités de VRAM (comme les 10 Go ou plus de la RTX 3080) au-delà de la limite d'adressage historique de 4 Go. ⁴
PCI Subsystem Settings	Re-Size BAR Support	Enabled ou Auto	Permet au CPU d'accéder à l'intégralité du framebuffer du GPU en une seule

			transaction, optimisant les performances graphiques.
Boot Configuration	CSM Support	Disabled	Le Compatibility Support Module doit être désactivé pour forcer un amorçage en UEFI pur. Son activation perturbe souvent le protocole GOP (Graphics Output Protocol), empêchant l'iGPU de s'initialiser comme affichage primaire. ⁵
Integrated Graphics Configuration	Initiate Graphic Adapter	IGD	Force la carte mère à initialiser le processeur graphique intégré (Radeon) en premier, garantissant que le processus de démarrage et le bootloader s'afficheront sur l'écran connecté à la carte mère. ⁶
Integrated Graphics Configuration	Integrated Graphics	Force	Empêche le firmware de désactiver l'iGPU lorsqu'une carte graphique dédiée est détectée sur le port PCIe. ⁶

L'activation du paramètre Above 4G Decoding est d'une importance capitale. Une carte

graphique moderne comme la NVIDIA RTX 3080 requiert un mappage mémoire (Memory-Mapped I/O ou MMIO) considérable. Si ce paramètre est inactif, le noyau Linux ou l'hyperviseur QEMU ne pourra pas allouer suffisamment d'espace d'adressage pour le périphérique lors de son transfert vers la machine virtuelle, ce qui se traduira par des erreurs d'allocation PCI (code 43 sous Windows ou erreurs vfio-pci dans les journaux système).⁴

Une fois ces modifications appliquées, sauvegardez les paramètres (généralement via la touche F10) et redémarrez la machine. L'écran connecté à la carte mère devrait afficher le logo MSI puis le chargeur d'amorçage de votre installation Arch Linux.

Phase 2 : Déploiement de l'Infrastructure Logicielle Hôte

L'environnement Arch Linux doit être doté des composants de virtualisation. L'écosystème repose sur KVM (le module noyau fournissant l'accélération matérielle), QEMU (l'émulateur matériel qui exécute la VM), et libvirt (l'API de gestion abstraite qui simplifie l'interaction avec QEMU via des configurations XML). virt-manager fournira l'interface graphique d'administration.

L'exécution de la commande pacman permet d'installer l'ensemble des dépendances strictes nécessaires à ce projet.⁷ La commande suivante récupère les paquets fondamentaux :

Bash

```
sudo pacman -Syu qemu-full libvirt virt-manager virt-viewer dnsmasq vde2 bridge-utils  
iptables-nft swtpm edk2-ovmf dkms linux-headers
```

Chaque composant installé joue un rôle critique dans l'architecture :

- **qemu-full** : Fournit l'hyperviseur complet avec le support de l'architecture x86_64 et de ses périphériques émulés.⁸
- **libvirt et virt-manager** : Le daemon de gestion et l'interface utilisateur graphique. Libvirt orchestre la traduction des paramètres XML en lignes de commande QEMU complexes.⁸
- **swtpm** : Un émulateur logiciel TPM (Trusted Platform Module). Windows 11 exigeant un TPM 2.0 pour son installation et son exécution, swtpm permet de fournir une enclave sécurisée virtuelle à la machine invitée sans nécessiter de puce TPM physique dédiée au passthrough.⁸
- **edk2-ovmf** : L'Open Virtual Machine Firmware. Il s'agit du firmware UEFI open-source pour les machines virtuelles. Il remplace le vieux BIOS SeaBIOS de QEMU, permettant à la machine virtuelle de démarrer en mode UEFI natif, une exigence pour Windows 11 et pour

le passthrough de périphériques PCIe modernes.¹

- **dkms et linux-headers** : L'infrastructure Dynamic Kernel Module Support. Ces paquets sont indispensables pour compiler ultérieurement le module kvmfr (nécessaire à Looking Glass) et les pilotes propriétaires NVIDIA à chaque mise à jour du noyau.¹¹

Afin de permettre à votre utilisateur courant de gérer les machines virtuelles et les réseaux virtuels sans recourir systématiquement à l'élévation de privilèges sudo, il est impératif de l'ajouter au groupe système libvirt.

Modifiez les groupes de votre utilisateur avec la commande suivante :

```
Bash
```

```
sudo usermod -aG libvirt $USER
```

Le service d'orchestration libvirtd doit ensuite être activé au démarrage du système et lancé immédiatement. Ce daemon est responsable de la création du réseau NAT par défaut (virbr0) et de l'écoute des requêtes de l'interface virt-manager.⁷

```
Bash
```

```
sudo systemctl enable --now libvirtd
```

Phase 3 : Paramétrage du Noyau et Validation de l'IOMMU

Pour que KVM puisse s'approprier un périphérique PCI physique, le noyau Linux de l'hôte doit être explicitement instruit d'activer le sous-système IOMMU d'AMD dès les premières phases de l'amorçage. Sans cela, le matériel refuse de dissocier les requêtes DMA de la carte graphique du reste du système.

L'objectif de cette phase n'est pas de capturer la RTX 3080 avec le pilote vfio-pci dès le démarrage (ce qui rendrait la carte inutilisable sous Linux), mais simplement de préparer le terrain pour un transfert dynamique ultérieur.³

Configuration du Gestionnaire d'Amorçage (Bootloader)

Deux arguments du noyau sont requis :

1. `amd_iommu=on` : Force l'activation de l'IOMMU spécifique aux processeurs AMD.¹³
2. `iommu=pt` : Le paramètre `pt` (passthrough) indique au noyau de ne pas traduire les requêtes DMA pour les périphériques qui ne sont pas explicitement assignés à une machine virtuelle. Cela améliore sensiblement les performances des périphériques hôtes (comme les contrôleurs NVMe ou USB) en évitant une surcharge de traduction inutile par l'IOMMU.⁶

La méthode d'injection de ces paramètres dépend de votre gestionnaire d'amorçage.

Si vous utilisez GRUB :

Éditez le fichier de configuration principal de GRUB.

```
Bash
```

```
sudo nano /etc/default/grub
```

Localisez la ligne `GRUB_CMDLINE_LINUX_DEFAULT` et ajoutez-y les paramètres :

```
GRUB_CMDLINE_LINUX_DEFAULT="... amd_iommu=on iommu=pt"
```

Générez ensuite la nouvelle configuration pour qu'elle soit prise en compte au prochain démarrage :

```
Bash
```

```
sudo grub-mkconfig -o /boot/grub/grub.cfg
```

Si vous utilisez systemd-boot (standard recommandé) :

Éditez le fichier de configuration de l'entrée Arch Linux située dans la partition EFI.

Bash

```
sudo nano /boot/loader/entries/arch.conf
```

Ajoutez les arguments à la fin de la ligne options :

```
options root=PARTUUID=XXXX rw amd_iommu=on iommu=pt
```

Une fois ces modifications enregistrées, redémarrez votre machine pour que le noyau charge la structure IOMMU.

Validation des Groupes IOMMU

Après le redémarrage, il est crucial de valider que la carte mère a correctement isolé la NVIDIA RTX 3080 dans son propre groupe. Un groupe IOMMU représente la plus petite unité de périphériques pouvant être transférée à une VM. Si la carte graphique partage son groupe avec un contrôleur SATA ou USB essentiel à l'hôte, le passthrough sera impossible ou provoquera un plantage du système.

Exécutez ce script bash qui parcourt le système de fichiers virtuel /sys pour lister la topologie IOMMU ⁷ :

Bash

```
#!/bin/bash
shopt -s nullglob
for g in /sys/kernel/iommu_groups/*; do
    echo "IOMMU Group ${g##*/}:"
    for d in $g/devices/*; do
        echo -e "\t$(lspci -nns ${d##*/})"
    done
done
```

Analysez la sortie et localisez votre RTX 3080. Vous devriez identifier deux périphériques distincts mais liés : le contrôleur VGA compatible (la puce graphique) et le contrôleur Audio (qui gère le son via HDMI/DisplayPort). Ces deux périphériques doivent appartenir au même groupe IOMMU. Sur l'architecture de la MSI MAG B850, le port PCIe x16 primaire est physiquement câblé au CPU, ce qui garantit une isolation parfaite.

Phase 4 : Gestion Graphique Hôte via Wayland et PRIME Offloading

L'environnement de bureau GNOME de l'hôte Arch Linux démarre nativement sur la session Wayland. L'écran étant connecté à la carte mère, le compositeur Mutter (le moteur de GNOME) s'initialise en utilisant l'iGPU AMD Radeon via le pilote libre amdgpu.

Cependant, pour que la RTX 3080 soit exploitable par l'hôte Linux (pour du rendu de modélisation 3D, de l'encodage NVENC ou des jeux natifs), les pilotes propriétaires NVIDIA doivent être installés et le mécanisme de rendu hybride PRIME (PRIME Render Offloading) doit être configuré.²

Le déploiement des pilotes s'effectue via pacman. L'utilisation de la version DKMS est recommandée pour assurer la pérennité du module lors des mises à jour régulières du noyau d'Arch Linux.

Bash

```
sudo pacman -S nvidia-dkms nvidia-utils lib32-nvidia-utils prime-run
```

Mécanique du PRIME Offloading sous Wayland

Historiquement complexe sous le serveur d'affichage Xorg (nécessitant des utilitaires tiers comme Bumblebee ou Optimus Manager), le rendu hybride est désormais nativement pris en charge par le noyau Linux et Wayland. La technologie PRIME permet d'indiquer au système qu'une application spécifique doit être exécutée sur le processeur graphique le plus puissant (le dGPU NVIDIA), bien que l'affichage final soit géré par le contrôleur graphique connecté à l'écran (l'iGPU AMD).

Le dGPU calcule les trames (frames) et les transfère via le bus PCIe directement dans la mémoire tampon (framebuffer) de l'iGPU, qui se charge de les envoyer au moniteur. Ce processus, appelé DMA-BUF sharing, s'effectue avec une perte de performance marginale.²

Le script prime-run installé précédemment est un simple wrapper qui injecte les variables d'environnement nécessaires pour forcer l'API graphique (OpenGL ou Vulkan) à cibler la carte NVIDIA.

Pour lancer une application (par exemple, Steam) sur la RTX 3080 sous Linux, exécutez simplement la commande précédée du wrapper :

Bash

prime-run steam

Tant que la commande prime-run n'est pas invoquée, le pilote NVIDIA maintient la RTX 3080 dans un état d'alimentation minimal (Power State D3), réduisant la consommation énergétique et la chauffe.¹⁵ C'est cet état d'inactivité qui rend possible le détachement dynamique du périphérique.

Phase 5 : L'Art du Détachement Dynamique via les Hooks Libvirt

Nous abordons ici la véritable prouesse architecturale de ce guide. Dans une configuration de passthrough classique (Single GPU), l'hyperviseur nécessite l'arrêt complet du gestionnaire d'affichage (GDM) pour libérer la carte graphique, ce qui ferme brutalement toutes les applications de l'utilisateur.

Notre objectif est d'exécuter un passthrough dynamique : la RTX 3080 est utilisée sous Linux, puis, au moment du lancement de la VM Windows 11, l'infrastructure libvirt doit intercepter la demande, purger les pilotes NVIDIA de la mémoire du noyau, attacher le pilote d'isolation vfio-pci, et démarrer la VM. Tout cela doit s'opérer en arrière-plan, pendant que l'utilisateur continue d'utiliser son bureau GNOME sur l'iGPU.³

Le défi majeur réside dans la gestion stricte du DRM (Direct Rendering Manager) par Wayland. Même si l'iGPU est le périphérique d'affichage primaire, le compositeur Wayland ou certains processus d'arrière-plan peuvent maintenir un descripteur de fichier ouvert sur la carte NVIDIA (/dev/nvidia0). Si un processus verrouille la carte, la commande de déchargement du module NVIDIA échouera silencieusement avec une erreur "module in use" (usage count non nul), provoquant le blocage de l'initialisation de la VM.³

Architecture des Scripts Libvirt (Hooks)

Libvirt dispose d'un mécanisme d'exécution de scripts (hooks) qui se déclenchent lors des changements d'état d'une machine virtuelle (démarrage, arrêt, migration).¹⁶

La première étape consiste à créer l'arborescence requise dans le répertoire de configuration de libvirt.

Bash

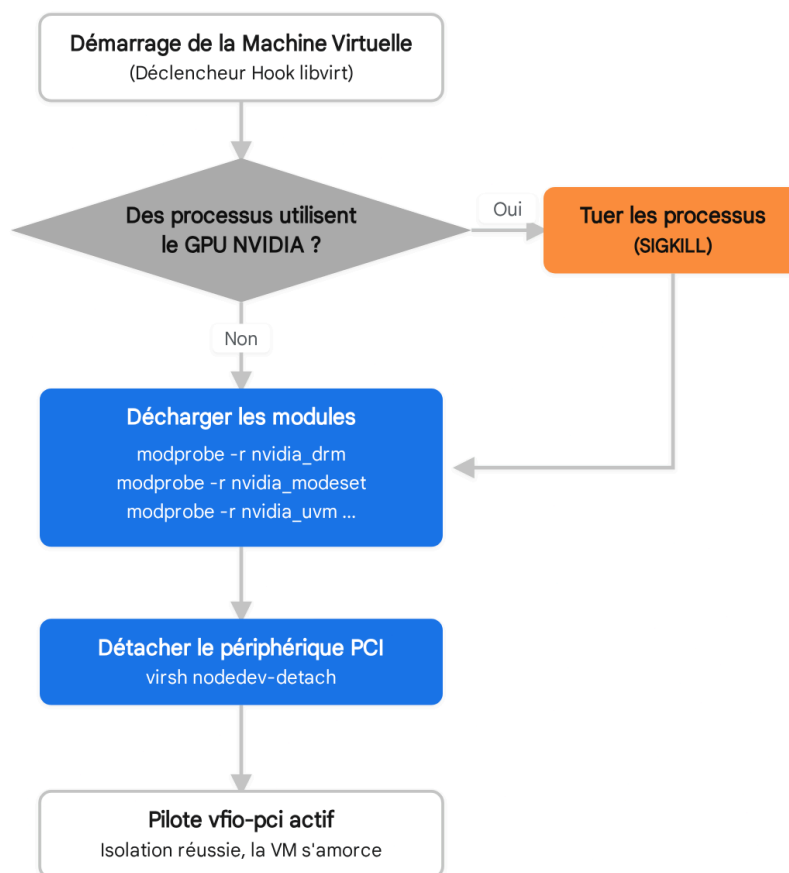
```
sudo mkdir -p /etc/libvirt/hooks/qemu.d/win11/prepare/begin  
sudo mkdir -p /etc/libvirt/hooks/qemu.d/win11/release/end
```

Libvirt ne lit qu'un seul fichier exécutable nommé qemu à la racine de /etc/libvirt/hooks/. Ce fichier doit agir comme un routeur (dispatcher) pour exécuter les sous-scripts pertinents en fonction de la VM et de son état. Téléchargez le gestionnaire de hook standard développé par la communauté VFIO :

Bash

```
sudo wget -O /etc/libvirt/hooks/qemu  
https://raw.githubusercontent.com/PassthroughPOST/VFIO-tools/master/libvirt\_hooks/qemu  
sudo chmod +x /etc/libvirt/hooks/qemu
```

Séquence d'Exécution du Hook de Détachement Dynamique



Le script intercepte la commande de démarrage de la VM, s'assure qu'aucune application Linux (via prime-run) n'exploite la RTX 3080, purge les modules propriétaires NVIDIA de la mémoire noyau, puis réassigne les adresses matérielles au pilote d'isolation vfio-pci.

Sources des données : [Reddit \(r/VFIO\)](#), [AskUbuntu](#)

Définition du Script de Démarrage (Prepare/Begin)

Le script de démarrage prépare l'hôte à se séparer de la carte graphique. Identifiez les adresses PCI exactes de votre RTX 3080. Si la commande `lspci -nn` indique que votre carte VGA est sur le bus 01:00.0 et l'Audio HDMI sur 01:00.1, ces adresses se traduisent dans l'interface libvirt par `pci_0000_01_00_0` et `pci_0000_01_00_1`.

L'éditeur de texte nano permet de créer le script exécuté juste avant l'initialisation de QEMU.¹⁹

Bash

```
sudo nano /etc/libvirt/hooks/qemu.d/win11/prepare/begin/start.sh
```

Le code source suivant a été méticuleusement conçu pour Wayland. Il évite la commande destructrice `systemctl stop gdm`, qui fermerait votre session.¹⁷ À la place, il utilise l'utilitaire `fuser` pour scruter les chemins des périphériques graphiques dans `/dev`. Si des processus résiduels (un jeu resté ouvert via `prime-run`) exploitent la carte, ils sont identifiés et terminés (`SIGKILL`).³ Le script ordonne ensuite le déchargement de la pile NVIDIA et l'attachement à `vfio-pci`.

Bash

```
#!/bin/bash
```

```
set -x
```

```
# Définition des adresses PCI de la RTX 3080 (VGA et Audio)
```

```
GPU_VGA="pci_0000_01_00_0"
```

```
GPU_AUDIO="pci_0000_01_00_1"
```

```
# 1. Vérification et purge des processus utilisant le dGPU
```

```
# Wayland ou des applications PRIME peuvent verrouiller le périphérique.
```

```
# /dev/nvidia* concerne les API CUDA/NVENC, /dev/dri/card1 concerne le rendu DRM.
```

```
if fuser -s /dev/nvidia* /dev/dri/card1; then
```

```
    echo "Fermeture forcée des processus exploitant la RTX 3080..."
```

```
    fuser -k -9 /dev/nvidia* /dev/dri/card1
```

```
    # Pause courte pour laisser le noyau libérer les descripteurs de fichiers
```

```
    sleep 2
```

```
fi
```

```
# 2. Déchargement séquentiel des modules propriétaires NVIDIA
```

```
# L'ordre est strict : le DRM dépend du modeset, qui dépend du module principal.
```

```
modprobe -r nvidia_drm
```

```
modprobe -r nvidia_modeset
```

```
modprobe -r nvidia_uvm
```

```
modprobe -r nvidia
```

3. Détachement logique des périphériques PCI du système hôte

```
virsh nodedev-detach $GPU_VGA
```

```
virsh nodedev-detach $GPU_AUDIO
```

4. Chargement de l'infrastructure d'isolation matérielle VFIO

```
modprobe vfio
```

```
modprobe vfio_pci
```

```
modprobe vfio_iommu_type1
```

Définition du Script d'Arrêt (Release/End)

Lorsque l'utilisateur éteint la machine virtuelle Windows 11, la libvirt invoque les scripts du répertoire release/end. L'objectif est d'inverser le processus : retirer le pilote VFIO pour rendre la carte matérielle au noyau Linux, et recharger les pilotes NVIDIA afin que la fonctionnalité prime-run soit de nouveau opérationnelle pour l'hôte.

Créez le script d'inversion :

```
Bash
```

```
sudo nano /etc/libvirt/hooks/qemu.d/win11/release/end/revert.sh
```

Insérez la logique de retour à l'état initial :

```
Bash
```

```
#!/bin/bash
```

```
set -x
```

```
GPU_VGA="pci_0000_01_00_0"
```

```
GPU_AUDIO="pci_0000_01_00_1"
```

1. Déchargement du pilote d'isolation

```
modprobe -r vfio_pci
```

```
modprobe -r vfio_iommu_type1
```

```
modprobe -r vfio
```

2. Réattachement des périphériques PCI au bus principal de l'hôte

```
virsh nodedev-reattach $GPU_VGA
```

```
virsh nodedev-reattach $GPU_AUDIO
```

3. Rechargement dynamique de la pile graphique NVIDIA

```
modprobe nvidia
```

```
modprobe nvidia_uvm
```

```
modprobe nvidia_modeset
```

```
modprobe nvidia_drm
```

Pour que la libvirt puisse exécuter ces actions, les droits d'exécution doivent être appliqués aux fichiers bash.

```
Bash
```

```
sudo chmod +x /etc/libvirt/hooks/qemu.d/win11/prepare/begin/start.sh
```

```
sudo chmod +x /etc/libvirt/hooks/qemu.d/win11/release/end/revert.sh
```

Phase 6 : Déploiement et Configuration de la Machine Virtuelle Windows 11

Avec l'infrastructure de transfert dynamique opérationnelle, il convient de configurer l'instance Windows 11. Microsoft impose des prérequis stricts pour son dernier système d'exploitation, notamment la présence d'une enclave sécurisée (TPM 2.0) et le démarrage via UEFI avec Secure Boot.

Lancez l'interface graphique virt-manager et démarrez l'assistant de création d'une nouvelle machine virtuelle, nommée "win11".

Personnalisation Pré-Installation du XML

Lors de la dernière étape de l'assistant de création, il est impératif de cocher l'option **"Customize configuration before install"**. Cette étape permet de modifier la définition matérielle avant le premier démarrage.

L'interface de configuration expose des composants virtuels qui doivent être ajustés avec une grande précision.

Section virt-manager	Paramétrage Requis	Explication Technique
Overview (Aperçu)	Firmware : Sélectionner le firmware OVMF avec support Secure Boot.	Le firmware UEFI (Open Virtual Machine Firmware) remplace le BIOS hérité. Windows 11 requiert l'UEFI. L'image OVMF_CODE.secboot.fd inclut les clés nécessaires pour l'amorçage sécurisé. ¹
Overview (Aperçu)	Chipset : Q35	Le chipset Q35 émule une architecture PCIe moderne, indispensable pour le passthrough d'une carte RTX 3080, contrairement au chipset i440FX limité au bus PCI standard.
CPUs	Modèle : host-model ou host-passthrough. Supprimer l'attribut migratable dans le XML.	La configuration <cpu mode='host-model' check='none'> sans la balise migratable empêche des conflits de mappage mémoire critiques lors de l'intégration ultérieure de la mémoire partagée IVSHMEM (Looking Glass), évitant ainsi un crash instantané de QEMU. ¹²
Add Hardware	TPM : Modèle CRB, Backend Emulated.	L'émulateur swtpm provisionne une puce Trusted Platform Module 2.0 logicielle, contournant la vérification matérielle stricte de l'installateur Windows 11. ⁸
Boot Options	Ajouter un lecteur CD-ROM	Les disques durs de la VM

	(SATA) avec l'image virtio-win.iso.	seront configurés sur le bus VirtIO (pour des performances d'E/S maximales). Windows ne possédant pas ces pilotes nativement, l'ISO fournit le pilote SCSI nécessaire lors de l'installation. ²¹
Add Hardware	PCI Host Device : Sélectionner la RTX 3080 et son contrôleur Audio.	Instruise QEMU d'intégrer les adresses matérielles réelles de la carte NVIDIA dans le domaine de la VM. Le script de hook configuré précédemment assurera la disponibilité de la carte à cet instant précis.

Lancez l'installation de Windows 11. Cette phase initiale s'affiche dans la fenêtre console classique de virt-manager (gérée par le protocole Spice). Lors du choix du disque cible, Windows ne détectera aucun lecteur. Chargez le pilote en naviguant dans le lecteur CD Virtio (viosstor/w11/amd64) pour révéler le disque dur virtuel NVMe/SCSI émulé, puis procédez à l'installation standard.²²

Une fois l'installation achevée, la machine virtuelle possède le contrôle matériel de la RTX 3080. Téléchargez les pilotes officiels NVIDIA depuis le site constructeur (évitez les utilitaires comme GeForce Experience, souvent capricieux en environnement virtualisé) et installez-les dans l'environnement Windows.¹²

Phase 7 : Intégration Visuelle à Faible Latence (Looking Glass)

À ce stade de la configuration, la RTX 3080 génère un flux vidéo Windows 11, mais ce signal est envoyé vers ses sorties physiques (HDMI/DisplayPort). Or, le cahier des charges impose l'utilisation d'un unique moniteur 4K branché sur la carte mère (iGPU).²³ Utiliser la fenêtre console QEMU standard induit une compression CPU destructrice pour les performances et une latence incompatible avec l'usage d'une RTX 3080.

La solution d'ingénierie avancée se nomme **Looking Glass**. Cette application capte les trames (frames) brutes et non compressées directement depuis le framebuffer mémoire de la RTX 3080 au sein de l'invité Windows, et les transfère à la vitesse de la mémoire vive vers le compositeur Wayland de l'hôte Arch Linux. Ce transfert s'effectue via un espace de mémoire

partagée sur le bus PCI, baptisé IVSHMEM (Inter-VM Shared Memory).¹²

Calcul et Allocation de la Mémoire KVMFR

Pour un affichage 4K ultrawide (3440x1440px), la taille du bloc mémoire partagée doit être rigoureusement calculée pour supporter le "Double Buffering" (stockage simultané de la trame en cours d'affichage et de la trame suivante en cours de rendu) afin d'éviter le déchirement de l'image (tearing).²⁵

La formule mathématique est la suivante : Largeur × Hauteur × 4 octets (profondeur de couleur RGBA) × 2 (buffers).

Calcul : $3440 \times 1440 \times 4 \times 2 = 39\,628\,800$ octets, soit environ 39,6 Mégaoctets.

Pour garantir l'alignement des pages mémoire du noyau, cette valeur doit être arrondie à la puissance de 2 supérieure, soit **64 Mo**.

Bien que QEMU puisse allouer ce bloc via le tmpfs standard (/dev/shm), l'approche la plus performante sous Arch Linux utilise un module noyau personnalisé nommé kvmfr (KVM FrameRelay). Ce module autorise les transferts DMA (Direct Memory Access) asynchrones de la mémoire partagée directement vers l'iGPU Radeon, réduisant l'utilisation du processeur central (CPU).²⁷

L'installation de l'application cliente et du module noyau s'effectue via l'AUR (Arch User Repository) :

```
Bash
```

```
yay -S looking-glass-rc kvmfr-dkms
```

La définition de la taille du tampon mémoire s'effectue en passant un argument au module kvmfr. Créez le fichier de configuration modprobe²⁸ :

```
Bash
```

```
sudo nano /etc/modprobe.d/kvmfr.conf
```

Ajoutez la ligne instruisant l'allocation statique de 64 Mo :

```
options kvmfr static_size_mb=64
```

Le module crée un fichier périphérique de caractères (/dev/kvmfr0). Pour que votre utilisateur puisse lire ce flux vidéo sans nécessiter les privilèges root, une règle udev s'impose.²⁷

Bash

```
sudo nano /etc/udev/rules.d/99-kvmfr.rules
```

Insérez la règle (remplacez votrenom par votre nom d'utilisateur) :

```
SUBSYSTEM=="kvmfr", OWNER="votrenom", GROUP="kvm", MODE="0660"
```

Chargez le module dans le noyau actif : `sudo modprobe kvmfr`.

Intégration IVSHMEM dans la Topologie de la VM

Le fichier périphérique hôte /dev/kvmfr0 doit maintenant être exposé à la machine virtuelle.

Ouvrez l'éditeur XML de libvirt pour votre VM :

Bash

```
virsh edit win11
```

Dans la section <devices>, injectez la définition du périphérique matériel partagé ¹² :

XML

```
<shmem name='looking-glass'>  
  <model type='ivshmem-plain'/>  
  <size unit='M'>64</size>  
</shmem>
```

Dans l'environnement Windows 11, l'installation se déroule en deux étapes :

1. Puisqu'aucun écran physique n'est branché sur la RTX 3080, la carte ne rendra aucune image. Il faut installer un pilote d'écran virtuel (Dummy Monitor). L'utilitaire open-source IddSampleDriver simule la présence d'un écran 4K matériellement connecté au GPU dédié.¹²
2. Installez l'application hôte (Host Application) de Looking Glass. Ce service système, lancé au démarrage de Windows, intercepte l'API Microsoft Desktop Duplication (DXGI) et copie chaque image vers l'espace mémoire IVSHMEM fraîchement créé.¹²

Optimisation du Client sous Wayland

L'affichage de la VM s'effectue désormais en lançant le client sur Arch Linux :

```
Bash
```

```
looking-glass-client -f /dev/kvmfr0
```

Un écueil technique fréquent survient sous les compositeurs Wayland (comme Mutter ou Hyprland) : l'API de rendu par défaut (EGL) provoque des artefacts visuels (scintillements noirs, corruption de fenêtres) lors de la mise à l'échelle. Pour y remédier, il faut forcer le client à utiliser le pipeline de rendu OpenGL standard.

Créez le fichier de configuration persistant du client ¹² :

```
Bash
```

```
nano ~/.looking-glass-client.ini
```

Paramétrez le rendu et liez l'interface à notre fichier mémoire :

```
Ini, TOML
```

```
[app]
```

```
renderer=opengl  
shmFile=/dev/kvmfr0
```

```
[win]  
fullScreen=yes
```

Le flux 3440x1440px généré par la RTX 3080 est désormais affiché en plein écran sur votre iGPU, avec une fluidité absolue de 60 Hz (ou plus, selon votre écran), procurant l'illusion parfaite d'une application Linux native.

Phase 8 : Intégration Ergonomique Totale (Presse-papiers et Virtio-FS)

Pour finaliser l'illusion d'un système unique, la frontière entre les environnements utilisateur doit être brisée. Le transfert de texte (presse-papiers) et le partage de fichiers volumineux doivent s'opérer instantanément, sans latence réseau.

Transparence du Presse-papiers via SPICE

Le protocole SPICE, bien que déprécié pour le transfert vidéo au profit de Looking Glass, demeure le standard industriel pour l'injection d'événements de saisie (clavier/souris) et la synchronisation du presse-papiers.

1. Dans virt-manager, assurez-vous que les périphériques Channel spice (Type: spicevmc) et Display Spice soient présents. Modifiez la configuration du périphérique d'affichage Spice pour définir son "Listen Type" sur None. Cette manipulation désactive la fenêtre vidéo classique tout en maintenant le canal de communication actif en arrière-plan.⁹
2. Dans Windows 11, exécutez l'installateur virtio-win-guest-tools.exe situé sur l'ISO Virtio.³³ Ce package déploie le démon SPICE vdaagent, qui s'exécute en tâche de fond et synchronise le presse-papiers de manière transparente entre le compositeur Wayland et l'API Windows.

Montage de Disque Natif via Virtio-FS

Le partage de dossiers via le réseau local virtuel (Samba/SMB) souffre d'un surcoût (overhead) lié à l'encapsulation TCP/IP, limitant les débits de transfert de gros volumes de données. La norme moderne est le protocole virtio-fs. Celui-ci s'appuie sur le framework FUSE (Filesystem in Userspace) côté hôte et utilise la fonctionnalité DAX (Direct Access). Le noyau invité contourne son propre cache mémoire pour lire directement les fichiers dans la mémoire physique allouée par l'hôte, garantissant des débits comparables à ceux d'un SSD NVMe local.³⁵

L'activation de virtio-fs exige un changement fondamental dans la topologie de la mémoire de la machine virtuelle : la RAM allouée (par exemple 16 Go) doit être déclarée comme une mémoire partagée (Shared Memory Backing), faute de quoi l'architecture DAX ne pourra pas

établir le pont entre les deux noyaux.

Éditez à nouveau le XML de la VM (virsh edit win11) et ajoutez le bloc de configuration mémoire partagée à la racine du domaine :

XML

```
<memoryBacking>  
  <access mode='shared' />  
</memoryBacking>
```

Dans l'interface graphique virt-manager, ajoutez un nouveau matériel de type **Filesystem** (Système de fichiers) :

- **Driver** : virtiofs
- **Source path** : Le chemin absolu du dossier hôte (ex: /home/archuser/PartageVM).
- **Target path** : Une étiquette d'identification arbitraire (ex: win_share).

Du côté du système invité Windows 11, la prise en charge de systèmes de fichiers personnalisés nécessite l'installation d'une infrastructure proxy, WinFSP, ainsi que la configuration d'un service système dédié à virtio-fs.³⁷

1. Téléchargez et installez l'utilitaire **WinFSP** (Windows File System Proxy). L'option d'installation minimale **Core** est suffisante pour activer le proxy système.³⁷
2. Le pilote matériel VirtIO FS Device doit apparaître comme fonctionnel dans le Gestionnaire de Périphériques Windows (il est inclus dans le déploiement préalable de virtio-win-guest-tools.exe).³³
3. Ouvrez une invite de commandes (cmd.exe) en bénéficiant des privilèges d'Administrateur. La commande sc.exe permet de créer et d'enregistrer le service Windows responsable du montage automatique du lecteur virtuel à chaque démarrage.³⁸

DOS

```
sc.exe create VirtioFsSvc binpath="C:\Program Files\Virtio-Win\VioFS\virtiofs.exe" start=auto  
depend="WinFsp.Launcher\VirtioFsDrv" DisplayName="Virtio FS Service"
```

Initiez le service pour monter le lecteur immédiatement :

DOS

sc `start` VirtioFsSvc

Le dossier système /home/archuser/PartageVM d'Arch Linux apparaît instantanément dans l'Explorateur Windows, généralement mappé sur la lettre de lecteur Z:. Ce lecteur permet la lecture et l'écriture de données sans aucune congestion réseau, scellant l'intégration absolue des deux environnements.³⁴

L'architecture déployée transcende la virtualisation classique. L'hôte Arch Linux préserve son environnement de travail immuable sous Wayland, capitalisant sur la stabilité de l'iGPU Radeon. La puissance de calcul de la NVIDIA RTX 3080 est exploitée avec une efficacité maximale : d'abord comme accélérateur de rendu pour le noyau Linux (PRIME), puis assignée dynamiquement via le système de hooks libvirt à l'instance Windows 11, sans aucune interruption du serveur d'affichage. Couplée au rapatriement vidéo à latence nulle de Looking Glass et à l'échange de fichiers natif de Virtio-FS, cette synergie technologique redéfinit le standard d'une station de travail polyvalente.

Sources des citations

1. PCI passthrough via OVMF - ArchWiki, consulté le février 25, 2026, https://wiki.archlinux.org/title/PCI_passthrough_via_OVMF
2. PRIME - ArchWiki, consulté le février 25, 2026, <https://wiki.archlinux.org/title/PRIME>
3. THE ULTIMATE SETUP: Dynamic GPU Unbinding: My Solution for Seamless VFIO Passthrough(I hope you can survive the rust glazing) - Reddit, consulté le février 25, 2026, https://www.reddit.com/r/VFIO/comments/1mp5z23/the_ultimate_setup_dynamic_gpu_unbinding_my/
4. What is option [Above 4G Decoding] in BIOS Setup? - MSI, consulté le février 25, 2026, <https://www.msi.com/faq/2726>
5. [MSI] Enable Or Disable Above 4G Decoding On BIOS MSI Motherboards - YouTube, consulté le février 25, 2026, <https://www.youtube.com/watch?v=vKrR81GeinQ>
6. Attempting to use igpu as main display and discrete gpu for ..., consulté le février 25, 2026, <https://bbs.archlinux.org/viewtopic.php?id=310537>
7. Marrca35/Single-GPU-Passthrough-for-Arch-Linux: This is ... - GitHub, consulté le février 25, 2026, <https://github.com/Marrca35/Single-GPU-Passthrough-for-Arch-Linux>
8. How to properly Install Windows 11 on QEMU/KVM Virtmanager on Arch linux with file sharing | 2025 - YouTube, consulté le février 25, 2026,

- <https://www.youtube.com/watch?v=woji50z1hF0>
9. virt-manager - ArchWiki, consulté le février 25, 2026,
<https://wiki.archlinux.org/title/Virt-manager>
 10. Running Windows 11 on Arch Linux with Virt-Manager TPM and Secure Boot (Silent Tutorial) - YouTube, consulté le février 25, 2026,
<https://www.youtube.com/watch?v=ik31RsNqMcw>
 11. Dynamic Kernel Module Support - ArchWiki, consulté le février 25, 2026,
https://wiki.archlinux.org/title/Dynamic_Kernel_Module_Support
 12. looking-glass-setup-guide.md · GitHub, consulté le février 25, 2026,
<https://gist.github.com/safwyls/96b6cf4b49e04af2668b7a77502e5ff2>
 13. [SOLVED] Issue with GPU Passthrough on Reboot / Newbie Corner / Arch Linux Forums, consulté le février 25, 2026,
<https://bbs.archlinux.org/viewtopic.php?id=292093>
 14. How do I use integrated graphics and a graphics card together on X? : r/VFIO - Reddit, consulté le février 25, 2026,
https://www.reddit.com/r/VFIO/comments/rbm881/how_do_i_use_integrated_graphics_and_a_graphics/
 15. NVIDIA Optimus: Cannot modprobe -r nvidia to dynamically bind GPU to vfio-pci driver, consulté le février 25, 2026,
https://www.reddit.com/r/VFIO/comments/m1c15h/nvidia_optimus_cannot_modprobe_r_nvidia_to/
 16. What are libvirt hooks bind/unbind drivers? : r/VFIO - Reddit, consulté le février 25, 2026,
https://www.reddit.com/r/VFIO/comments/jsjc8a/what_are_libvirt_hooks_bindunbind_drivers/
 17. Detaching GPU from Wayland compositor doesn't work properly - NVIDIA Developer Forums, consulté le février 25, 2026,
<https://forums.developer.nvidia.com/t/detaching-gpu-from-wayland-compositor-doesnt-work-properly/317128>
 18. bryansteiner/gpu-passthrough-tutorial - GitHub, consulté le février 25, 2026,
<https://github.com/bryansteiner/gpu-passthrough-tutorial>
 19. Can't bind/unbind GPU from nvidia to vfio-pci properly on demand without reboot (Ubuntu 22 QEMU KVM OVMF), consulté le février 25, 2026,
<https://askubuntu.com/questions/1509972/cant-bind-unbind-gpu-from-nvidia-to-vfio-pci-properly-on-demand-without-reboot>
 20. PRIME GPU Passthrough / Laptop Issues / Arch Linux Forums, consulté le février 25, 2026, <https://bbs.archlinux.org/viewtopic.php?id=268844>
 21. Virtualize Windows 11 with QEMU on Arch Linux, consulté le février 25, 2026,
https://std.rocks/virtualization_qemu_windows11.html
 22. Windows 11 guest best practices - Proxmox VE, consulté le février 25, 2026,
https://pve.proxmox.com/wiki/Windows_11_guest_best_practices
 23. Performance | Looking Glass Light Field Docs, consulté le février 25, 2026,
<https://docs.lookingglassfactory.com/keyconcepts/3d-design-guidelines/performance>
 24. Display Settings on Windows | Looking Glass Light Field Docs, consulté le février

- 25, 2026,
<https://docs.lookingglassfactory.com/software/looking-glass-bridge/display-settings-on-windows>
25. Installation — Looking Glass B4 documentation - GitHub Pages, consulté le février 25, 2026, <https://jjrcop.github.io/lg-sphinx-demo/install/>
 26. Looking Glass - Dev Oops, consulté le février 25, 2026, <https://mccgillij.dev/looking-glass-vfio.html>
 27. IVSHMEM with the KVMFR module (Recommended) — Looking Glass B7 documentation, consulté le février 25, 2026, https://looking-glass.io/docs/B7/ivshmem_kvmfr/
 28. Looking-glass kvmfr setup Arch - GitHub Gist, consulté le février 25, 2026, <https://gist.github.com/Ruakij/dd40b3d7cac5d0f196d1116771b6e42>
 29. Installation — Looking Glass B6 documentation, consulté le février 25, 2026, <https://looking-glass.io/docs/B6/install/>
 30. I've finally done it. : r/VFIO - Reddit, consulté le février 25, 2026, https://www.reddit.com/r/VFIO/comments/11sdj5b/i_have_finally_done_it/
 31. Client usage — Looking Glass B7 documentation, consulté le février 25, 2026, <https://looking-glass.io/docs/B7/usage/>
 32. virt-manager - ArchWiki, consulté le février 25, 2026, <https://wiki.archlinux.org/title/Virt-manager#Virtio-FS>
 33. TUTORIAL: Configuring VirtioFS for a Windows Server 2025 Guest on Proxmox 8.4 - Reddit, consulté le février 25, 2026, https://www.reddit.com/r/homelab/comments/1k9l4ea/tutorial_configuring_virtiofs_for_a_windows/
 34. Sharing Files between the Linux Host and a Windows VM using virtiofs - Heiko's Blog, consulté le février 25, 2026, <https://www.heiko-sieger.info/sharing-files-between-the-linux-host-and-a-windows-vm-using-virtiofs/>
 35. How do I set-up file sharing between my host(arch) & guest(windows)? : r/VFIO - Reddit, consulté le février 25, 2026, https://www.reddit.com/r/VFIO/comments/wpw1bj/how_do_i_setup_file_sharing_between_my_hostarch/
 36. Chapter 20. Sharing files between the host and its virtual machines - Red Hat Documentation, consulté le février 25, 2026, https://docs.redhat.com/it/documentation/red_hat_enterprise_linux/10/html/configuring_and_managing_windows_virtual_machines/sharing-files-between-the-host-and-its-virtual-machines
 37. Virtiofs: Shared file system · virtio-win/kvm-guest-drivers-windows Wiki - GitHub, consulté le février 25, 2026, <https://github.com/virtio-win/kvm-guest-drivers-windows/wiki/Virtiofs:-Shared-file-system>
 38. How to install virtiofs drivers on Windows, consulté le février 25, 2026, <https://virtio-fs.gitlab.io/howto-windows.html>